

Instructions:

- When writing python code be careful about spacing / indents. Leave enough room that it is obvious when you want things indented.
- You should pull off the first pages of the exam to use as reference.
- Make sure to write legibly.
- Please write your name on the top of the real exam page.
- You will have one hour to complete the exam.
- You are allowed to use commands not covered in class, but if they do not work as expected on my computer they will not receive full credit.

Example Flask App

```
from flask import Flask, request, jsonify, Response
import os

app = Flask(__name__)

@app.route('/api/students/<string:student_name>', methods=['GET'])
def get_student_major(student_name):
    student_majors = {"sarah": "computer_science", "mike": "mathematics", "emma": "computer_science"}
    result_dict = {}
    if student_name in student_majors:
        result_dict[student_name] = student_majors[student_name]
        return jsonify(result_dict), 200

    return jsonify({"error": "Student not found"}), 404

@app.route('/api/students/<string:student_name>/info', methods=['GET'])
def get_student_info(student_name):
    student_data = registrar_api_call(student_name)
    # Returns a tuple of the form: (name (string), year (int), gpa (float))
    # Ex: ("Sarah", 3, 3.75)
    # If the student does not exist it returns None

    # CODE GOES HERE

@app.route('/api/students/<int:student_id>', methods=['GET'])
def get_student_by_id(student_id):
    api_key = os.environ.get('API_KEY')
    if not api_key:
        return jsonify({"error": "API key missing"}), 500
    return jsonify({"id": student_id, "key_set": True}), 200

@app.route('/api/students', methods=['POST'])
def create_student():
    data = request.get_json()
    if not data:
        return jsonify({"error": "No JSON data"}), 400
    name = data.get('name')
    if not name:
        return jsonify({"error": "Name required"}), 400
    return jsonify({"message": f"Created {name}"}), 201
```

```
/
├── Dockerfile
├── makefile
├── pyproject.toml
├── app
│   └── app.py
├── log_files
│   ├── 2025.03.11_app.log
│   ├── 2025.03.12_app.log
│   ├── 2025.03.13_app.log
│   ├── 2025.03.14_app.log
│   ├── 2025.03.15_app.log
│   └── 2025.03.16_app.log
├── files
│   ├── archive
│   │   ├── file1.pdf
│   │   ├── file2.pdf
│   │   ├── image1.png
│   │   └── image2.png
│   └── processed
│       └── final.pdf
```

```
FROM astral/uv:python3.13-bookworm
```

```
WORKDIR /project
COPY pyproject.toml .
```

```
RUN uv venv
RUN uv sync
```

```
CMD ["uv", "run", "python", "app/app.py"]
```

Makefile Example

```
IMAGE_NAME=student_api

.PHONY=build interactive run

build:
    docker build . -t $(IMAGE_NAME)

interactive: build
    docker run -it -v $(shell pwd):/project/files $(IMAGE_NAME) /bin/bash

run: build
    docker run -p 4000:5000 -v $(shell pwd):/project/files $(IMAGE_NAME)
```

Name: _____

Group / Peer Review

Please list the names of your project group members and indicate what percentage of the work each person contributed. **The total percent should be roughly 100%.** If there are any comments you would like to state about your group dynamics, please use this space. You are welcome, but not required to add notes.

Member Name (include yourself) Percentage of Work	

Additional notes or comments:

1. Using **bash** commands please complete the following. You are currently in the `/log_files` directory. The files contain logs generated by a custom logger, similar to what was demonstrated in class. Each file contains logs from the date specified in the file name. You can see an example of the format of the logs in the sample below.

Example Log File

```
2025-03-15 14:22:18 | ERROR | ERROR205 File Format
2025-03-15 14:22:45 | INFO | GET request received at /api/students/sarah/info
2025-03-15 14:23:10 | INFO | GET request received at /api/students/123
2025-03-15 14:23:35 | ERROR | GET request received at /api/students/john
2025-03-15 14:24:01 | ERROR | ERROR205 File Format
2025-03-15 14:24:15 | ERROR | ERROR042 Database Connection
2025-03-15 14:24:18 | WARNING | Retrying database connection
2025-03-15 14:24:22 | INFO | Database connection restored
2025-03-15 14:24:45 | INFO | GET request received at /api/students/mike/info
2025-03-15 14:25:10 | WARNING | Rate limit approaching
2025-03-15 14:25:30 | WARNING | High memory usage detected
2025-03-15 14:26:15 | ERROR | ERROR042 Database Connection
```

- (a) (5 Points) Using **bash** commands, create a file (`warnings.log`, in the current directory) which contains all lines that have **WARNING** written in them (this should be *case sensitive*) from the file `2025.03.15_app.log`. E.g. we want a file which contains only those lines.

- (b) (5 Points) Using **bash** commands, print to the screen the *number* of lines that contain **ERROR042** in the log file `2025.03.15_app.log`.

- (c) (5 Points) Some log entries incorrectly contain both **ERROR** and **WARNING** in the same line. For example:

```
2025-03-15 14:24:18 | ERROR | WARNING: some problem
```

Using **bash** commands, print to the screen the number of lines in `2025.03.15_app.log` that contain **ERROR** but do *not* contain **WARNING**.

2. (5 Points) When we run `make interactive` what is the full path, *inside the container*, for the file `final.pdf`?
3. (2 Points) Fill in the blank: `requirements.txt` is to `pip` as _____ is to `uv`.
4. (5 Points) Your current working directory is `/log_files`, using a *relative* path and a single command (do not change directories) please list all files that end with `.pdf` in the `files/archive` directory.
5. Looking at our flask server code above, please write what is returned when the following calls are made. Write both the body and the status code.
 - (a) (3 Points) A GET request to `/api/students/mike`
 - (b) (3 Points) A GET request to `/api/students/david`
6. (1 Point) What does DRY stand for when speaking about code quality?

7. (5 Points) The code for `/api/students/<string:student_name>/info` needs to be completed (see above). The code should return:

- If the student exists it should return a JSON object of the form:
`{"name": NAME OF STUDENT, "year": YEAR OF STUDENT}` with a status code of 200.
- If the student does not exist it should return an error message in JSON format (you can decide what it says) with a status code of 404.
- You do not need to write any lines already present in the template above.

8. (5 Points) Please write a unit test (using `assert`) for the following function. This function only needs to have one test condition. Please name the function `test_calculate_score`.

```
def calculate_score(base, multiplier, bonus):  
    return (base * multiplier) + bonus
```

9. (2 Points) What is the difference between an LLM and an AI Agent? (3 sentences or less)

10. (2 Points) What is the primary difference between Docker and Docker Compose? (1 sentence)

11. (5 Points) We want to create a function that returns a piecewise function. A piecewise function $f(x)$ can be defined mathematically as:

$$\text{piecewise}(x) = \begin{cases} \text{positive}(x) & \text{if } x \geq \text{cutoff} \\ \text{negative}(x) & \text{if } x < \text{cutoff} \end{cases}$$

where cutoff is the breakpoint, positive is the function used for values greater than or equal to the breakpoint, and negative is the function used for values less than the breakpoint.

Please write a function `piecewise` which takes in three arguments: (1) `positive_func`, (2) `negative_func`, and (3) `cutoff`. The function should return a new function. When this returned function is called with `value`, it should return `positive_func(value)` if `value ≥ cutoff`, otherwise it should return `negative_func(value)`. You can assume that both `positive_func` and `negative_func` only take a single argument.

For example, if `square` squares a number and `abs` returns the absolute value, then `result_func = piecewise(square, abs, 0)` should create a function such that `result_func(5)` returns 25 and `result_func(-3)` returns 3.

12. (5 Points) We want to set an `API_KEY` environment variable for our Flask application running in Docker. There are two approaches we are considering:

- (a) Add `ENV API_KEY=abc123` to the Dockerfile
- (b) Pass `-e API_KEY=abc123` when running `docker run`

Explain the key difference between these two approaches. What are the practical implications of this difference? (2-3 sentences)

13. (2 Points) When using autodocs systems like MkDocs, what is the primary part of the Python code that is automatically extracted and included in the documentation? (One sentence or less)

14. (2 Points) What is the difference between a unit test and an integration test? (1-2 sentences)

15. (2 points) What does MCP stand for?

16. (2 points) Explain how an MCP would be used (2-3 sentences).
17. (5 Points) Write a function called `apply_functions` that takes a list of functions and a value, and returns a list of results from applying each function to the value in order. For example, if you have functions `square` (which squares a number) and `double` (which doubles a number), calling `apply_functions([square, double], 5)` should return `[25, 10]`.
18. (5 Points) Write a function called `apply_to_all` that takes a list of functions and a list of values, and returns a list of results. For each value in the input list, apply all functions to that value and collect the results. The function should return a list of lists, where each inner list contains the results of applying all functions to one value. For example, if you have functions `square` and `double`, calling `apply_to_all([square, double], [2, 3])` should return `[[4, 4], [9, 6]]` (`square(2)=4`, `double(2)=4` for first value; `square(3)=9`, `double(3)=6` for second value).

19. (5 Points) Write a Flask route `/api/students/grades` that accepts a POST request. The request body should contain JSON with a `student_id` (required) and any number of course grades as key-value pairs (e.g., `{"student_id": 123, "math": 85, "science": 92}`).

Assume there is a helper function `validate_student_id(student_id)` that returns `True` if the student ID is valid, and `False` otherwise. Use this function to validate the `student_id`. If validation fails, return status 400 with an error message.

If validation passes, the route should:

- Extract all course grades (all keys except `student_id`)
- Return a JSON response with status 200 containing: `{"student_id": ID, "courses": [LIST OF COURSE NAMES], "grades": [LIST OF GRADE VALUES]}`

For example, if the input is `{"student_id": 123, "math": 85, "science": 92}`, the response should be `{"student_id": 123, "courses": ["math", "science"], "grades": [85, 92]}`.

20. (5 Points) Please write a function `logger_helper` that accepts a function parameter named `func` as input. You can assume that `func` accepts zero arguments. The function should act similar to a logging decorator, but simpler. It should use `print` to print `"Function XXX started"` before the function starts, making sure to substitute the name of the function for `XXX`, and then should print `"Function Complete"` when the function ends. This does not need to use Python's logging module or any specific wraps functionality.